

GstPipeline

A `GstPipeline` ([gststreamer/gstpipeline.html#GstPipeline](#)) is a special `GstBin` ([gststreamer/gstbin.html#GstBin](#)) used as the toplevel container for the filter graph. The `GstPipeline` ([gststreamer/gstpipeline.html#GstPipeline](#)) will manage the selection and distribution of a global `GstClock` ([gststreamer/gstclock.html#GstClock](#)) as well as provide a `GstBus` ([gststreamer/gstbus.html#GstBus](#)) to the application.

`gst_pipeline_new` ([gststreamer/gstpipeline.html#gst_pipeline_new](#)) is used to create a pipeline. when you are done with the pipeline, use `gst_object_unref` ([gststreamer/gstobject.html#gst_object_unref](#)) to free its resources including all added `GstElement` ([gststreamer/gstelement.html#GstElement](#)) objects (if not otherwise referenced).

Elements are added and removed from the pipeline using the `GstBin` ([gststreamer/gstbin.html#GstBin](#)) methods like `gst_bin_add` ([gststreamer/gstbin.html#gst_bin_add](#)) and `gst_bin_remove` ([gststreamer/gstbin.html#gst_bin_remove](#)) (see `GstBin` ([gststreamer/gstbin.html#GstBin](#))).

Before changing the state of the `GstPipeline` ([gststreamer/gstpipeline.html#GstPipeline](#)) (see `GstElement` ([gststreamer/gstelement.html#GstElement](#))) a `GstBus` ([gststreamer/gstbus.html#GstBus](#)) should be retrieved with `gst_pipeline_get_bus` ([gststreamer/gstpipeline.html#gst_pipeline_get_bus](#)). This `GstBus` ([gststreamer/gstbus.html#GstBus](#)) should then be used to receive `GstMessage` ([gststreamer/gstmessage.html#GstMessage](#)) from the elements in the pipeline. Listening to the `GstBus` ([gststreamer/gstbus.html#GstBus](#)) is necessary for retrieving error messages from the `GstPipeline` ([gststreamer/gstpipeline.html#GstPipeline](#)) and otherwise the `GstPipeline` ([gststreamer/gstpipeline.html#GstPipeline](#)) might stop without any indication, why. Furthermore, the `GstPipeline` ([gststreamer/gstpipeline.html#GstPipeline](#)) posts messages even if nobody listens on the `GstBus` ([gststreamer/gstbus.html#GstBus](#)), which will pile up and use up memory.

By default, a `GstPipeline` ([gststreamer/gstpipeline.html#GstPipeline](#)) will automatically flush the pending `GstBus` ([gststreamer/gstbus.html#GstBus](#)) messages when going to the NULL state to ensure that no circular references exist when no messages are read from the `GstBus` ([gststreamer/gstbus.html#GstBus](#)). This behaviour can be changed with `gst_pipeline_set_auto_flush_bus` ([gststreamer/gstpipeline.html#gst_pipeline_set_auto_flush_bus](#)).

When the `GstPipeline` ([gststreamer/gstpipeline.html#GstPipeline](#)) performs the PAUSED to PLAYING state change it will select a clock for the elements. The clock selection algorithm will by default select a clock provided by an element that is most upstream (closest to the source). For live pipelines (ones that return `GST_STATE_CHANGE_NO_PREROLL` ([gststreamer/gstelement.html#GST_STATE_CHANGE_NO_PREROLL](#)) from the `gst_element_set_state` ([gststreamer/gstelement.html#gst_element_set_state](#)) call) this will select the clock provided by the live source. For normal pipelines this will select a clock provided by the sinks (most likely the audio sink). If no element provides a clock, a default `GstSystemClock` ([gststreamer/gstsystemclock.html#GstSystemClock](#)) is used.

The clock selection can be controlled with the `gst_pipeline_use_clock` ([gststreamer/gstpipeline.html#gst_pipeline_use_clock](#)) method, which will enforce a given clock on the pipeline. With `gst_pipeline_auto_clock` ([gststreamer/gstpipeline.html#gst_pipeline_auto_clock](#)) the default clock selection algorithm can be restored.

A `GstPipeline` ([gststreamer/gstpipeline.html#GstPipeline](#)) maintains a running time for the elements. The running time is defined as the difference between the current clock time and the base time. When the pipeline goes to READY or a flushing seek is performed on it, the running time is reset to 0. When the pipeline is set from PLAYING to PAUSED, the current clock time is sampled and used to configure the base time for the elements when the pipeline is set to PLAYING again. The effect is that the running time (as the difference between the clock time and the base time) will count how much time was spent in the PLAYING state. This default behaviour can be changed with the `gst_element_set_start_time` ([gststreamer/gstelement.html#gst_element_set_start_time](#)) method.

GstPipeline

```
GObject (https://docs.gtk.org/gobject/class.Object.html)
├── GInitiallyUnowned (https://docs.gtk.org/gobject/class.InitiallyUnowned.html)
│   ├── GstObject (gststreamer/gstobject.html#GstObject)
│   │   ├── GstElement (gststreamer/gstelement.html#GstElement)
│   │   │   ├── GstBin (gststreamer/gstbin.html#GstBin)
│   │   │   │   └── GstPipeline
```

The `GstPipeline` ([gststreamer/gstpipeline.html#GstPipeline](#)) structure.

Members

`bin` (`GstBin` ([gststreamer/gstbin.html#GstBin](#))) – No description available

`fixed_clock` (`GstClock` ([gststreamer/gstclock.html#GstClock](#)) *) – The fixed clock of the pipeline, used when `GST_PIPELINE_FLAG_FIXED_CLOCK` is set.

stream_time (GstClockTime (gststreamer/gstclock.html#GstClockTime)) – The stream time of the pipeline. A better name for this property would be the *running_time*, the total time spent in the PLAYING state without being flushed. (deprecated, use the *start_time* on GstElement).

delay (GstClockTime (gststreamer/gstclock.html#GstClockTime)) – Extra delay added to *base_time* to compensate for computing delays when setting elements to PLAYING.

Class structure

GstPipelineClass

Fields

parent_class (GstBinClass (GstBinClass)) – No description available

Constructors

gst_pipeline_new

```
GstElement (gststreamer/gstelement.html#GstElement) *
gst_pipeline_new (const gchar (https://docs.gtk.org/glib/types.html#gchar) * name)
```

Create a new pipeline with the given name.

Parameters:

name ([nullable (NULL may be passed to the value)]) – name of new pipeline

Returns ([transfer: floating (Alias for transfer none, used for objects with floating refs)]) – newly created GstPipeline MT safe.

Methods

gst_pipeline_auto_clock

```
gst_pipeline_auto_clock (GstPipeline (gststreamer/gstpipeline.html#GstPipeline) * pipeline)
```

Let *pipeline* select a clock automatically. This is the default behaviour.

Use this function if you previous forced a fixed clock with *gst_pipeline_use_clock* (gststreamer/gstpipeline.html#gst_pipeline_use_clock) and want to restore the default pipeline clock selection algorithm.

MT safe.

Parameters:

pipeline – a GstPipeline (gststreamer/gstpipeline.html#GstPipeline)

gst_pipeline_get_auto_flush_bus

```
gboolean (https://docs.gtk.org/glib/types.html#gboolean)
gst_pipeline_get_auto_flush_bus (GstPipeline (gststreamer/gstpipeline.html#GstPipeline) * pipeline)
```

Check if *pipeline* will automatically flush messages when going to the NULL state.

Parameters:

pipeline – a GstPipeline (gststreamer/gstpipeline.html#GstPipeline)

Returns – whether the pipeline will automatically flush its bus when going from READY to NULL state or not. MT safe.

gst_pipeline_get_bus

```
GstBus (gststreamer/gstbus.html#GstBus) *
gst_pipeline_get_bus (GstPipeline (gststreamer/gstpipeline.html#GstPipeline) * pipeline)
```

Gets the GstBus (gststreamer/gstbus.html#GstBus) of *pipeline*. The bus allows applications to receive GstMessage (gststreamer/gstmessage.html#GstMessage) packets.

Parameters:

pipeline – a GstPipeline (gststreamer/gstpipeline.html#GstPipeline)

Returns ([transfer: full (Free data after the code is done)]) – a GstBus (gststreamer/gstbus.html#GstBus), unref after usage. MT safe.

gst_pipeline_get_clock

```
GstClock (gststreamer/gstclock.html#GstClock) *
gst_pipeline_get_clock (GstPipeline (gststreamer/gstpipeline.html#GstPipeline) * pipeline)
```

Gets the current clock used by *pipeline*. Users of object oriented languages should use `gst_pipeline_get_pipeline_clock` (gststreamer/gstpipeline.html#gst_pipeline_get_pipeline_clock) to avoid confusion with `gst_element_get_clock` (gststreamer/gstelement.html#gst_element_get_clock) which has a different behavior.

Unlike `gst_element_get_clock` (gststreamer/gstelement.html#gst_element_get_clock), this function will always return a clock, even if the pipeline is not in the PLAYING state.

Parameters:

pipeline – a GstPipeline (gststreamer/gstpipeline.html#GstPipeline)

Returns ([transfer: full (Free data after the code is done)]) – a GstClock (gststreamer/gstclock.html#GstClock), unref after usage.

gst_pipeline_get_configured_latency

```
GstClockTime (gststreamer/gstclock.html#GstClockTime)
gst_pipeline_get_configured_latency (GstPipeline (gststreamer/gstpipeline.html#GstPipeline) * pipeline)
```

Return the configured latency on *pipeline*.

Parameters:

pipeline – a GstPipeline (gststreamer/gstpipeline.html#GstPipeline)

Returns – *pipeline* configured latency, or GST_CLOCK_TIME_NONE (gststreamer/gstclock.html#GST_CLOCK_TIME_NONE) if none has been configured because *pipeline* did not reach the PLAYING state yet. MT safe.

Since : 1.24

gst_pipeline_get_delay

```
GstClockTime (gststreamer/gstclock.html#GstClockTime)
gst_pipeline_get_delay (GstPipeline (gststreamer/gstpipeline.html#GstPipeline) * pipeline)
```

Get the configured delay (see `gst_pipeline_set_delay` (gststreamer/gstpipeline.html#gst_pipeline_set_delay)).

Parameters:

pipeline – a GstPipeline (gststreamer/gstpipeline.html#GstPipeline)

Returns – The configured delay. MT safe.

gst_pipeline_get_latency

```
GstClockTime (gststreamer/gstclock.html#GstClockTime)
gst_pipeline_get_latency (GstPipeline (gststreamer/gstpipeline.html#GstPipeline) * pipeline)
```

Gets the latency that should be configured on the pipeline. See `gst_pipeline_set_latency` ([gststreamer/gstpipeline.html#gst_pipeline_set_latency](#)).

Parameters:

pipeline – a `GstPipeline` ([gststreamer/gstpipeline.html#GstPipeline](#))

Returns – Latency to configure on the pipeline or `GST_CLOCK_TIME_NONE`

Since : 1.6

gst_pipeline_get_pipeline_clock

```
GstClock (gststreamer/gstclock.html#GstClock) *
gst_pipeline_get_pipeline_clock (GstPipeline (gststreamer/gstpipeline.html#GstPipeline) * pipeline)
```

Gets the current clock used by *pipeline*.

Unlike `gst_element_get_clock` ([gststreamer/gstelement.html#gst_element_get_clock](#)), this function will always return a clock, even if the pipeline is not in the PLAYING state.

Parameters:

pipeline – a `GstPipeline` ([gststreamer/gstpipeline.html#GstPipeline](#))

Returns ([transfer: full (Free data after the code is done)]) – a `GstClock` ([gststreamer/gstclock.html#GstClock](#)), unref after usage.

Since : 1.6

gst_pipeline_is_live

```
gboolean (https://docs.gtk.org/glib/types.html#gboolean)
gst_pipeline_is_live (GstPipeline (gststreamer/gstpipeline.html#GstPipeline) * pipeline)
```

Check if *pipeline* is live.

Parameters:

pipeline – a `GstPipeline` ([gststreamer/gstpipeline.html#GstPipeline](#))

Returns – TRUE (<https://web.mit.edu/barnowl/share/gtk-doc/html/glib/glib-Standard-Macros.html#TRUE:CAPS>) if *pipeline* is live, FALSE (<https://web.mit.edu/barnowl/share/gtk-doc/html/glib/glib-Standard-Macros.html#FALSE:CAPS>) if not or if it did not reach the PAUSED state yet. MT safe.

Since : 1.24

gst_pipeline_set_auto_flush_bus

```
gst_pipeline_set_auto_flush_bus (GstPipeline (gststreamer/gstpipeline.html#GstPipeline) * pipeline,
                                gboolean (https://docs.gtk.org/glib/types.html#gboolean) auto_flush)
```

Usually, when a pipeline goes from READY to NULL state, it automatically flushes all pending messages on the bus, which is done for recounting purposes, to break circular references.

This means that applications that update state using (async) bus messages (e.g. do certain things when a pipeline goes from PAUSED to READY) might not get to see messages when the pipeline is shut down, because they might be flushed before they can be dispatched in the main thread. This behaviour can be disabled using this function.

It is important that all messages on the bus are handled when the automatic flushing is disabled else memory leaks will be introduced.

MT safe.

Parameters:

pipeline – a GstPipeline (gststreamer/gstpipeline.html#GstPipeline)

auto_flush – whether or not to automatically flush the bus when the pipeline goes from READY to NULL state

gst_pipeline_set_clock

```
gboolean (https://docs.gtk.org/glib/types.html#gboolean)
gst_pipeline_set_clock (GstPipeline (gststreamer/gstpipeline.html#GstPipeline) * pipeline,
                       GstClock (gststreamer/gstclock.html#GstClock) * clock)
```

Set the clock for *pipeline*. The clock will be distributed to all the elements managed by the pipeline.

Parameters:

pipeline – a GstPipeline (gststreamer/gstpipeline.html#GstPipeline)

clock ([transfer: none (Don't free data after the code is done)][nullable (NULL may be passed to the value)]) – the clock to set

Returns – TRUE (https://web.mit.edu/barnowl/share/gtk-doc/html/glib/glib-Standard-Macros.html#TRUE:CAPS) if the clock could be set on the pipeline. FALSE (https://web.mit.edu/barnowl/share/gtk-doc/html/glib/glib-Standard-Macros.html#FALSE:CAPS) if some element did not accept the clock. MT safe.

gst_pipeline_set_delay

```
gst_pipeline_set_delay (GstPipeline (gststreamer/gstpipeline.html#GstPipeline) * pipeline,
                       GstClockTime (gststreamer/gstclock.html#GstClockTime) delay)
```

Set the expected delay needed for all elements to perform the PAUSED to PLAYING state change. *delay* will be added to the base time of the elements so that they wait an additional *delay* amount of time before starting to process buffers and cannot be GST_CLOCK_TIME_NONE (gststreamer/gstclock.html#GST_CLOCK_TIME_NONE).

This option is used for tuning purposes and should normally not be used.

MT safe.

Parameters:

pipeline – a GstPipeline (gststreamer/gstpipeline.html#GstPipeline)

delay – the delay

gst_pipeline_set_latency

```
gst_pipeline_set_latency (GstPipeline (gststreamer/gstpipeline.html#GstPipeline) * pipeline,
                          GstClockTime (gststreamer/gstclock.html#GstClockTime) latency)
```

Sets the latency that should be configured on the pipeline. Setting GST_CLOCK_TIME_NONE will restore the default behaviour of using the minimum latency from the LATENCY query. Setting this is usually not required and the pipeline will figure out an appropriate latency automatically.

Setting a too low latency, especially lower than the minimum latency from the LATENCY query, will most likely cause the pipeline to fail.

Parameters:

pipeline – a GstPipeline (gststreamer/gstpipeline.html#GstPipeline)

latency – latency to configure

Since : 1.6

gst_pipeline_use_clock

```
gst_pipeline_use_clock (GstPipeline (gststreamer/gstpipeline.html#GstPipeline) * pipeline,
                       GstClock (gststreamer/gstclock.html#GstClock) * clock)
```

Force *pipeline* to use the given *clock*. The pipeline will always use the given clock even if new clock providers are added to this pipeline.

If *clock* is NULL (<https://web.mit.edu/barnowl/share/gtk-doc/html/glib/glib-Standard-Macros.html#NULL:CAPS>) all clocking will be disabled which will make the pipeline run as fast as possible.

MT safe.

Parameters:

pipeline – a GstPipeline ([gstreamer/gstpipeline.html#GstPipeline](https://gstreamer.gstpipeline.html#GstPipeline))

clock ([\[transfer: none \(Don't free data after the code is done\)\]\[allow-none \(NULL is OK, both for passing and returning\)\]](#)) – the clock to use

Properties

auto-flush-bus

“auto-flush-bus” gboolean (<https://docs.gtk.org/glib/types.html#gboolean>)

Whether or not to automatically flush all messages on the pipeline's bus when going from READY to NULL state. Please see `gst_pipeline_set_auto_flush_bus` ([gstreamer/gstpipeline.html#gst_pipeline_set_auto_flush_bus](https://gstreamer.gstpipeline.html#gst_pipeline_set_auto_flush_bus)) for more information on this option.

Flags : Read / Write

delay

“delay” guint64 (<https://docs.gtk.org/glib/types.html#guint64>)

The expected delay needed for elements to spin up to the PLAYING state expressed in nanoseconds. see `gst_pipeline_set_delay` ([gstreamer/gstpipeline.html#gst_pipeline_set_delay](https://gstreamer.gstpipeline.html#gst_pipeline_set_delay)) for more information on this option.

Flags : Read / Write

latency

“latency” guint64 (<https://docs.gtk.org/glib/types.html#guint64>)

Latency to configure on the pipeline. See `gst_pipeline_set_latency` ([gstreamer/gstpipeline.html#gst_pipeline_set_latency](https://gstreamer.gstpipeline.html#gst_pipeline_set_latency)).

Flags : Read / Write

Since : 1.6

Function Macros

GST_PIPELINE_CAST

```
#define GST_PIPELINE_CAST(obj) ((GstPipeline*)(obj))
```

Enumerations

GstPipelineFlags

Pipeline flags

Members

`GST_PIPELINE_FLAG_FIXED_CLOCK` (524288) – this pipeline works with a fixed clock

`GST_PIPELINE_FLAG_LAST` (8388608) – offset to define more flags

